

Inżynieria wiedzy i symboliczne uczenie maszynowe

Prolog

Mateusz Jarosz
mateusja@agh.edu.pl

Prolog - PROgramowanie w LOGice

język programowania - jakby się ktoś jeszcze nie domyślił :)

jest językiem deklaratywnym czyli nie zapisujemy ciągu zdarzeń prowadzących do rozwiązania a raczej jak wygląda problem a silnik prologa sam ma wywnioskować rozwiązanie.

Stałe

stałe ciąg liter i znaków rozpoczynający się od małej litery

- ser
- p1P
- fd
- ewqr42335323553525332523532532
- ~~1ewfsafsadfas2er~~
- ~~PdsaeweadsaPsad~~

Liczby są szczególnymi rodzajami stałych

- 10
- 3.14
- 1e-5

Zmienne

ciąg liter i znaków rozpoczynający się od dużej litery

- Ser
- P1
- SDASD
- DSArqw
- ...

podkreślenie oznacza dowolną nieistotną wartość _

Termy - nie nie te łącznie na słowacji

w prologu termy to inaczej wyrażenia mogą być one proste jak i złożone:

proste to stałe i zmienne

złożone jak można się domyślić uzyskujemy łącząc wiele prostych lub/i złożonych w jedną

takie łączenie odbywa się przy użyciu funktorów podobnie jak zmienne zapisujemy je jako ciągi liter i cyfr rozpoczynające się z małej litery.

- `para(a, b)`
- `para(a, para(b, c))`

Unifikacja

podstawowa operacja wykonywana na termach

rozstrzyga ona czy dwa termy są identyczne, jeżeli jeden z termów to zmienna może również zajść konieczność podstawienia

- $S = T$

przykłady:

- $a = b$
- $a = X$
- $\text{para}(a, b) = \text{para}(X, Y)$
- $\text{para}(a, b) = \text{para}(X, X)$

Predykaty

wyrażają one pewne warunki.

wbudowane:

- `var(T)`
- `nonvar(T)`
- `atom(T)`
- `number(T)`
- `compound(T)`

co to zrobisz?

$\text{para}(a, b) = \text{para}(X, Y), Z = \text{para}(Y, X).$

$X = a; X = b.$

$P = \text{para}(123, a), (P = \text{para}(E, _); P = \text{para}(_, E)), \backslash + \text{number}(E).$

Predykaty cd.

możemy również definiować własne predykaty

predykaty zapisujemy w formacie klauzul każda klauzula musi być zakończona kropką i zawiera fakty lub reguły

Fakty:

```
rodzice(zeus,hera,atena)
```

```
rodzice(zeus,leto,apollo)
```

reguły

```
ojciec(X,Y) :- rodzice(X, _, Y)
```

```
matka(X,Y) :- rodzice(_, X, Y)
```

zagadka 2

`element(X, para(X, _)).`

`element(X, para(_, X)).`

`P = para(a, para(b, c)), element(E, P), element(E2, E).`

co będzie pod E i E2

Listy

lista składa się z pierwszego elementu listy głowy oraz z całej reszty ogona

`[]` - stała oznaczająca listę pustą

`.` (kropka) funktor łączący głowę z ogonem

np. `.(a,[])` (`.` zdaje się nie działać w każdej implementacji prologa)

druga bardziej naturalna opcja

`[a]`

`[a,b]`

`[a,[b,c]]`

`[a|[[b,c]]]`

zagadka 3

$s(0, 1).$

$s(N, X) :- N1 \text{ is } N - 1, s(N1, X1), X \text{ is } N * X1.$

Cięcie !

jest to Prologowa operacja, którą można umiejscowić w każdym miejscu klauzuli, zapobiega ona dalszemu wykonaniu (cofnięciu i wyszukaniu innych rozwiązań, backtracking)

$a(X) :- b(X), c(X), !, d(X), e(X).$

lepszy przykład

$\text{max}(A,B,B) :- A < B, !.$

$\text{max}(A,B,A).$

ćwiczenie

zdefiniuj predykat:

1. elem sprawdzający czy element należy do listy
2. długość l zwracający długość listy

Dynamiczne dodawanie predykatów

`assert(X)` - dodaje fakt albo klauzule do bazy, fakt/ klauzula jest ewaluowana jako ostatnia dla danego klucza

`asserta(X)` - ???

`assertz(X)` - ???

Dynamiczne dodawanie predykatów - przykład

```
:-dynamic good/2.
```

```
:-dynamic bad/2.
```

```
assert(good(skywalker, luke)).
```

```
assert(good(solo, han)).
```

```
assert(bad(vader, darth)).
```

```
listing(good).
```

Dynamiczne usuwanie predykatów

`retract(X)` - usuwa predykat `X` z bazy

`retractall(X)` - usuwa wszystkie predykaty z bazy których głowa(head) unifikuje się z `X`

Dynamiczne usuwanie predykatów - przykład

```
retract(bad(vader, darth)).
```

```
retractall(good(_, _)).
```

```
good(X, Y).
```

ćwiczenie

Zaimplementuj prosty system ekspertowy pozwalający określić czy dana sieć jest bezpieczna, wnioskowanie powinno opierać się o co najmniej 5 reguł i 10 faktów.

```
% Facts about the network
network status(up).
ping sweep detected.
port_scan_detected.
```

```
% Rule for network status
network secure :-
    network_status(up),
    not ping_sweep_detected(yes),
    not port_scan_detected(yes),
    not ddos_attack_detected(yes).
```

ale ale jak to włączyć

swiprolog - jest wersja portable

fakty i reguły zapisujemy w pliku .pl

w konsoli kompilujemy plik [nazwa_pliku] i możemy używać reguł w zapytaniach

```
main :- write('This is sample Prolog program'),write(' This program is written into  
hello_world.pl file').
```

proszę spróbować to umieścić w pliku hello.pl w konsoli wpisujemy [hello]. a następnie main.

można też korzystać z toola online <https://swish.swi-prolog.org/>

albo <https://logicalenglish.logicalcontracts.com/>